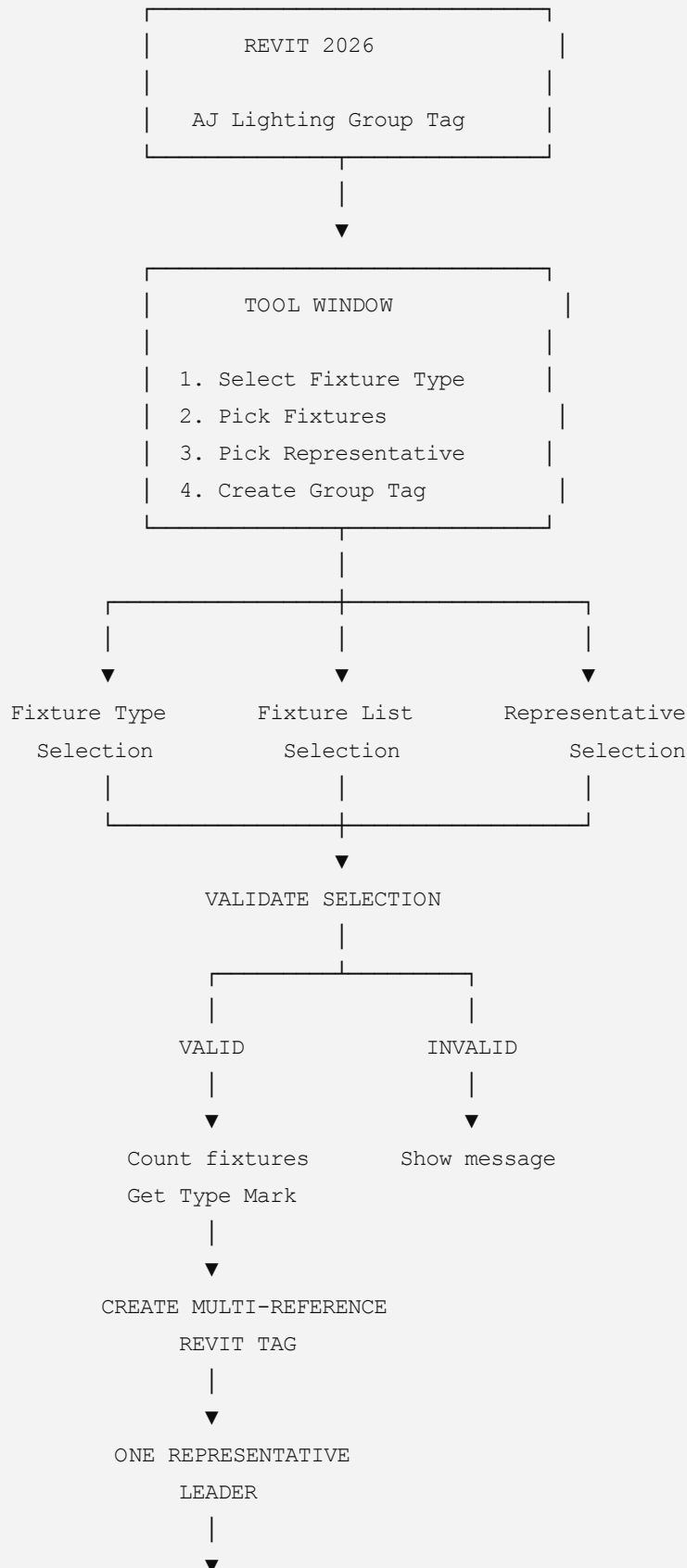


Absolutely. Since we're now going **directly to a Revit 2026 API add-in**, here is the complete **UI + O/I architecture** you can hand to **Antigravity** as the implementation specification.

AJ Lighting Group Tag — Revit 2026

1. Overall workflow



FREE END



(4) EL_01

2. Main UI

I recommend a **modeless WPF window** so the user can interact with Revit while the tool is open.

AJ Lighting Group Tag	[X]
FIXTURE TYPE	
EL_01 ▼	
Available Types:	
• EL_01 • EL_02 • EL_03 • EL_04	
STEP 1 - SELECT FIXTURES	
[PICK FIXTURES]	
Selected: 4	
✓ Fixture 01	EL_01
✓ Fixture 02	EL_01
✓ Fixture 03	EL_01
✓ Fixture 04	EL_01
STEP 2 - REPRESENTATIVE	
[PICK REPRESENTATIVE]	
Representative: Fixture 02	

GROUP INFORMATION

Type Mark: EL_01
Number of Fixtures: 4
Leader: ONE
Leader Target: Fixture 02
Leader End: FREE

[RESET]

[CREATE TAG]

3. Input / Output architecture

This is the important part for Antigravity.

INPUT 1 — Fixture Type

User chooses a Revit **Family Type**.

Example:

Lighting Fixture
└─ EL_01

Input

SelectedFamilySymbol

Data

ElementId
FamilyName
TypeName
TypeMark

Example:

FamilyName = AJ_Lighting
TypeName = EL_01
TypeMark = EL_01

4. INPUT 2 — Fixture Selection

User clicks:

```
[PICK FIXTURES]
```

Then Revit enters selection mode.

The user can select fixtures individually:

```
Fixture 01  
Fixture 02  
Fixture 03  
Fixture 04
```

Then finish/cancel the selection operation.

Internal data

```
List<FamilyInstance>
```

Example:

```
Fixtures =  
[  
    Fixture_001,  
    Fixture_002,  
    Fixture_003,  
    Fixture_004  
]
```

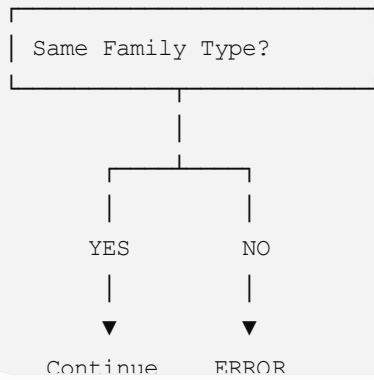
Output

```
SelectedFixtureCount = 4
```

5. Fixture validation

The tool should validate **before creating the tag**.

```
Selected Fixtures  
|  
▼
```



Validation:

Check 1

All selected elements must be:

FamilyInstance

Check 2

All must be:

OST_LightingFixtures

Check 3

All must have the same:

FamilySymbol.Id

or, depending on the intended grouping rule:

Type Mark

For your current workflow, I would use **same Family Type**.

Check 4

At least one fixture must be selected.

6. INPUT 3 — Representative Fixture

After the group has been selected:

[PICK REPRESENTATIVE]

User clicks **one fixture**.

Example:

Selected group:

F1

F2

F3

F4

Representative:

F2

Internal data

RepresentativeFixture

Validation

The representative **must belong to the selected fixture list**.

Representative.Id

↓

Is it in Fixtures?

↓

YES → Continue

NO → Error

This is important because the leader should point to one of the fixtures represented by the tag.

7. GROUP DATA OBJECT

Once selection is complete, create one internal object:

LightingGroup

Conceptually:

LightingGroup

|

├─ FamilySymbol

|

```
|— FamilyName
|
|— TypeName
|
|— TypeMark
|
|— Fixtures[]
|
|— Count
|
|— Representative
|
|— View
|
|— Tag
```

Example:

```
LightingGroup
|
|— FamilyName = AJ_Lighting
|— TypeName = EL_01
|— TypeMark = EL_01
|— Count = 4
|
|— Fixtures
|   |— F1
|   |— F2
|   |— F3
|   └— F4
|
|— Representative = F2
```

8. TAG CREATION

The Revit API layer then receives:

```
CreateGroupTag (
    View,
    Fixtures,
    Representative,
    TagType
)
```

Input

```
View
Fixtures[]
Representative
TagFamily / TagType
```

Output

```
IndependentTag
```

9. Multi-reference tag logic

The tag is initially created against the representative:

```
Representative
|
▼
IndependentTag
```

Then the remaining references are added:

```
Tag
|
├─ F1
├─ F2 ← Representative
├─ F3
└─ F4
```

The tag therefore represents:

```
4 × EL_01
```

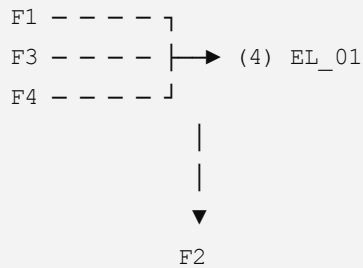
10. ONE LEADER logic

This is critical.

The tool should configure the tag so that:

```
F1 — hidden leader
F2 — VISIBLE leader
F3 — hidden leader
F4 — hidden leader
```


Conceptually:



Only **F2**, the representative, has the visible leader.

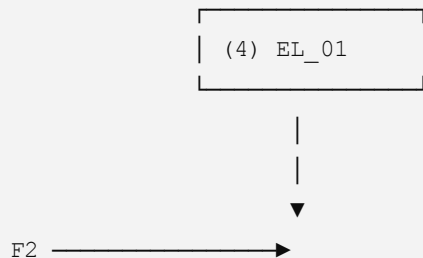
11. Free leader

The leader must be:

```
LeaderEndCondition.Free
```

So the user can move the leader endpoint independently.

Target behavior:



The tag head can be moved while the representative leader remains controllable.

12. Tag family

The tool should search for:

```
AJ_Lighting_Group_Tag
```

If found:

```
Use tag type
```

If not:

ERROR

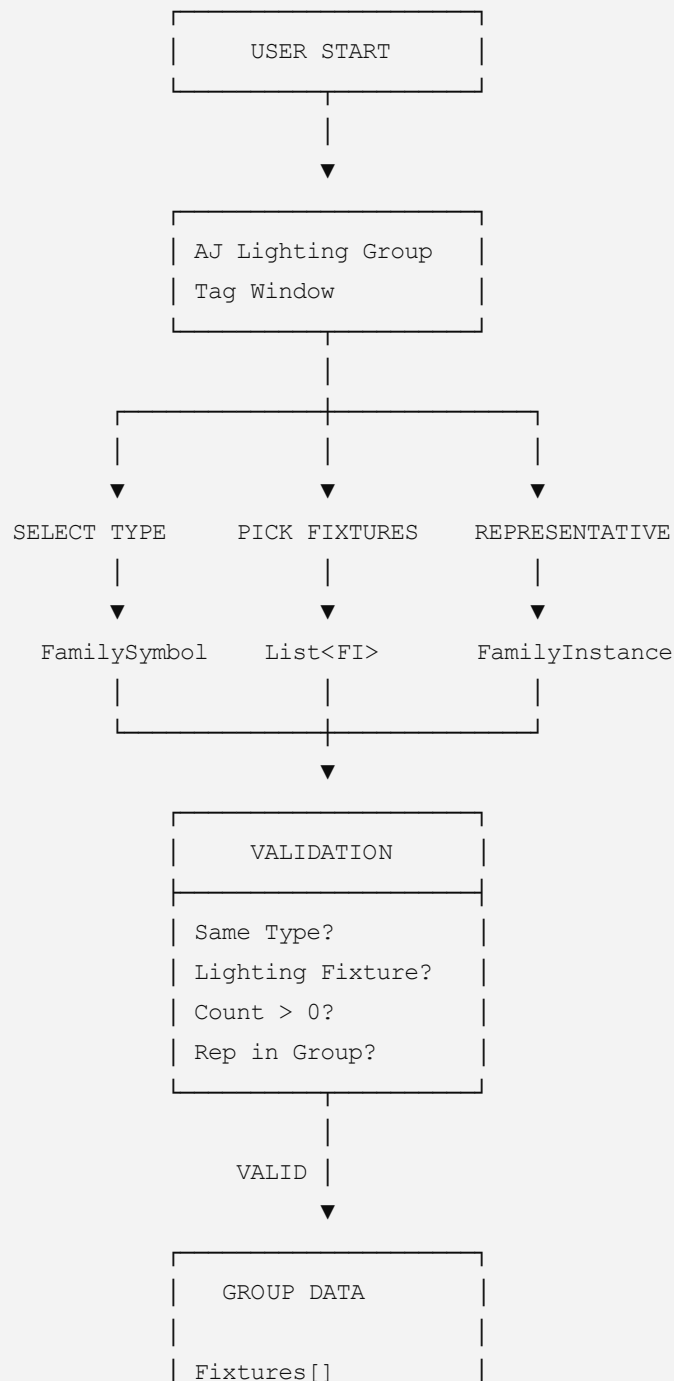
AJ_Lighting_Group_Tag
was not found in this project.

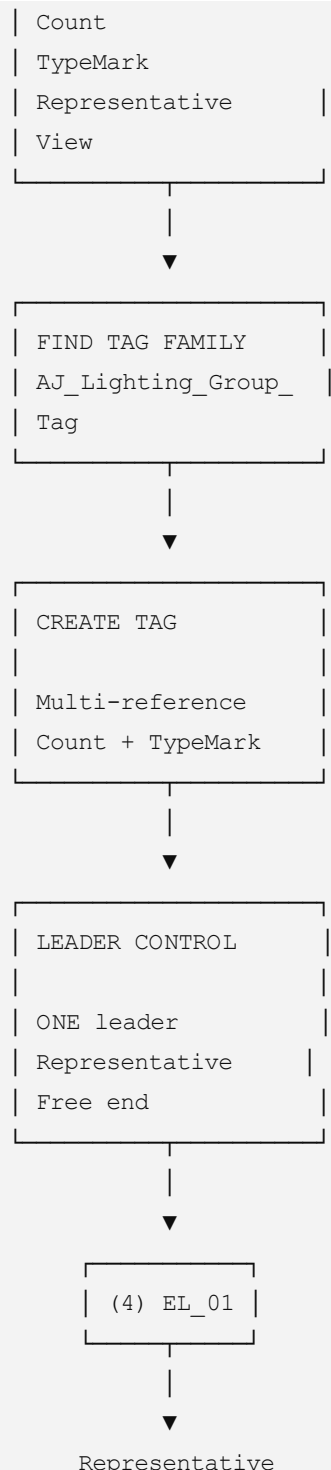
Please load the tag family.

Don't silently substitute another tag family.

13. Final O/I diagram

This is the diagram I would give Antigravity as the **master implementation diagram**:





14. Antigravity implementation modules

I'd structure the Revit 2026 `.NET 8` project like this:

```

AJ.LightingGroupTag
|
├── Commands
│   └── LaunchLightingGroupTagCommand.cs
|
├── UI
│   ├── LightingGroupTagWindow.xaml
│   └── LightingGroupTagWindow.xaml.cs
|
└── Models
  
```

```

|   └─ LightingGroup.cs
|
|─ Selection
|   └─ FixtureSelectionService.cs
|   └─ RepresentativeSelectionService.cs
|
|─ Validation
|   └─ FixtureGroupValidator.cs
|
|─ Tags
|   └─ LightingGroupTagService.cs
|
|─ Revit
|   └─ RevitElementService.cs
|
└─ Resources
    └─ AJ_Lighting_Group_Tag.rfa

```

Clean separation

```

UI
|
▼
Selection Service
|
▼
Validation Service
|
▼
LightingGroup Model
|
▼
Tag Service
|
▼
Revit API

```

This gives Antigravity a very clear target: **the UI handles the user workflow; the Revit API services handle selection, validation and tag creation; the `LightingGroup` object carries the data between them.**

And importantly, we're no longer trying to make Dynamo do something it isn't particularly good at. This is now a proper **Revit 2026 automation tool**, which is exactly the direction you originally wanted.